

Guide - MLAGentBench user manual

This guide is specially designed to provide control flow over the procedure for reproducing the work assembled within this Benchmark, under new editions of the original docker image from:

<https://github.com/snap-stanford/MLAgentBench/>.

Within these editions, the benchmark was molded and prepared to deal with local models such as **llama-3.1-8B** and **Qwen-2.5-7B**. In addition to the implementation of traces such as CodeCarbon to control and monitor energy consumption. This function acts as a flag to be activated or deactivated. By default, it is deactivated if you wish to perform the tests without this tool.

With that said, a new Dockerfile was generated applying all updates and new installations to help with the use of the new additions, which can be found in the following repository:

<https://github.com/JDgr12/MLAgentBench>

Some Features of the Dockerfile

- **Image Size:** ~45GB
- **Installations:*** Based on the original image and its main libraries
<qhwang123/researchassistant:latest>
- **2 Requirements files:** Requirements_main.txt and Requirements_vllm_srv.txt
- **Environments:** Conda activate / deactivate autogpt & vllm_srv
- For simplicity, the container will start immediately in autogpt.

Recomendations to be aware

- **Carefully consider which models you wish to use and make the relevant changes explained in the document.**
- **Be very clear about the folders where the results are saved and where to save some of the plots/Excel files.**
- **This work is designed to view and compare the performance of two models through two response formats:**
 - Rigid: Thought, Action, Action input
 - Original (Proposed by MLAGentBench): Reflection, Research Plan and Status, Fact check, Thought, Action, Action input

- It is set by default to respond rigidly, in order to minimise model hallucination. If you wish to change it back to the original setting, please first refer to section 7 of the configuration of 'agent_research.py' before performing any task tests.

Assembly steps

1. The Setting Up the (Host)

- **Clone repository:** The following command can be applied in the work folder where cloning is to be performed

```
git clone https://github.com/JDgr12/MlAgentBench.git
cd MlAgentBench
```

- **Credential management:** It is essential to configure the API keys before launching the container.
 - **OpenAI:** The benchmark initially worked with this API key to determine its normal behavior, but it was kept to prevent warnings and searches performed by the program. There is no need for any expense; the generated key is sufficient (for free), as long as the file is in the same path as all the files when you do the git clone it will work.

```
echo "org-YOUR_ORG_KEY:sk-XXXX_YOUR_API_KEY" > openai_api_key.txt
```

If you want to ensure that the file was saved correctly with your API key

```
cat openai_api_key.txt
```

- - **Kaggle:** It is important to have a Kaggle API key, given that several of the tasks worked on within MlAgentBench are Kaggle datasets/challenges. To do this, it is important to have a .kaggle file inserted in the same directory as the other cloned files. It should be added that each of the Kaggle challenges must be accepted. These are: **house-price, spaceship-titanic, amp-parkinson-disease-progression-prediction, fathomnet, feedback and identify-contrails**

```
mkdir -p ~/.kaggle # if you want to have it on home "system"
cp /path/to/your/kaggle.json ~/.kaggle/kaggle.json
```

```
mkdir -p .kaggle #if you want to have it inside the MlAgentBench folder
nano kaggle.json # to create the file kaggle.json inside ".kaggle"
```

If you are using VS Code, it is easier to simply put the Kaggle API key inside .kaggle
I do recomend to move the Kaggle api inside just for precautionary reasons
If you want to se you kaggle api key

```
cat .kaggle/kaggle.json # Inside, it has to be like this.  
# {"username":"YOUR_USERNAME","key":"YOUR_API_KEY"}
```

2. Container deployment

- **Built the image:**

```
docker build -t mlagentbench-thesis . # You can name your image as you wish
```

- **Launch the container:** exposing the ports for the API (8001) and the inference server (8002).

```
docker run --rm --gpus all --user root -w /MLAgentBench \  
--name mlagentbench-thesis-ctr \  
-p 8001:8000 \  
-p 8002:8002 \  
-v ${PWD}:/MLAgentBench \  
-it mlagentbench-thesis # this is the name of your image
```

NOTE: This command uses --rm so that when you exit the container, it is deleted. We recommend not using it if it is going to be used for a long time, so that modifications, installations, and more importantly configurations (kaggle) are not lost. It also comes with --user root, which allows the user to edit, install, create/delete files, etc.

3. Inference Server (vLLM Backend)

In a new bash, we will re-enter the container and activate the environment for our backend where our local LLM will be located.

```
docker exec -it mlagentbench-thesis-ctr bash # New entry to the active container  
conda activate vll_srv
```

- **Huggingface permissions:** Since the models are used by Hugging Face, you must first access the page accept the license, request rights and permissions for use. Obtain an API token from your Hugging Face account to log in to the container. That is why it is not recommended to start the container with --rm, as you would have to repeat this process several times.

We log in to the container, it will ask us to insert the token (not visible in bash), and we will answer **n** to the question it will ask us about adding the token as git credentials (it is not very relevant to answer Y/n), but once configured within this non-erasable container, it does not have to be done again.

```
huggingface-cli login
# Right before insert your Token
```

Here are the links for each model

[Llama-3.1-8B-Instruct](#)

[Qwen-2.5-7B-Instruct](#)

Once both models have been approved for use, we can apply the following commands to mount them:

LLama 3.1-8B

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-8B-Instruct \
  --served-model-name llama-3.1-8B-Instruct \
  --port 8002 \
  --tensor-parallel-size 1 \
  --trust-remote-code
```

Qwen 2.5-7B

```
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-7B-Instruct \
  --served-model-name qwen2.5-7b-instruct \
  --port 8002 \
  --tensor-parallel-size 1 \
  --max-model-len 4096 \
  --gpu-memory-utilization 0.9 \
  --trust-remote-code
```

As you can see, Qwen's command is somewhat different due to space limitations on the L40S device, given that with that configuration it will only use 40GB of the device's VRAM, it is free to modify gpy memeory utilization.

4. Task Execution (Agent Runner)

- **Prepare any task:** 1. Cifar10, imdb, ogbn-arxiv, house-price, spaceship-titanic, amp-parkinson-disease-progression-prediction, fathomnet, feedback, identify-contrails, CLRS, babylm,

vectorization

- Due to file density issues, tasks do not come with the **prepare** command once loaded, as GitHub has a maximum data upload capacity.

```
python -u -m MLAGentBench.prepare_task <task_name> $(which python)
```

In the case of a Kaggle task, it will ask you to press any key, and it will download all the information. The image already has the file unzip library, so it will do this automatically. Once each task is complete, it will tell you, "prepared."

If you are not sure whether that command has already been run, when you run it again it will say "prepare.py not found or already prepared."

- **Direct execution test:** This section is only intended for testing the individual operation of a task, for example, cifar10 with the following commands:

```
#We delete any unnecessary files as a precaution and recreate the <folder_name>
# And workspace folders.
#You are free to choose the name of the folder at this point.
```

```
rm -rf <folder_name> workspace && mkdir -p <folder_name> workspace
```

Here in this command, we are indicating which working folder we are going to use, where we are going to save the results, enable the use of CodeCarbon, use the GPU, and specify the models we will use to ensure consistency with the backend.

```
python -u -m MLAGentBench.runner \
--python "$(which python)" \
--task cifar10 \
--device 0 \
--log-dir <folder_name> \
--work-dir workspace \
--llm-name llama-3.1-8B-Instruct \
--fast-llm-name llama-3.1-8B-Instruct \
--edit-script-llm-name llama-3.1-8B-Instruct \
--use-codecarbon \
> <folder_name>/log 2>&1
```

Now we can apply the evaluation of that task with the model, just to see if it generated the **submission.csv** file, which will tell us what value it obtained and how long it took to obtain it.

```
python -m MAgentBench.eval \
  --log-folder <folder_name> \
  --task cifar10 \
  --output-file <folder_name>/resultados_llm.json
```

Inside this folder "folder_name", we will find several other folders: **agent_log** (which stores the agent's actions and main log), **codecarbon** (which stores the .csv files obtained from the agent), **env_log** (which parses and identifies the actions taken by the agent from the environment's .csv files, the tool_logs on the actions executed, the traces, where each of the steps taken in the task can be seen, and finally the trace.json with the information from the env_log) and finally we will see the **main log** and the **resultados_llm.json** with all the information by steps and energy consumption.

5. Automation with JMeter and FastAPI

This section is where the actual tests and large-scale results are carried out to assess the behaviour, performance, variability, and success of the model being evaluated.

NOTE: Depending on the model you have decided to use and mount in the backend in vllm_srv, you need to review the mlagentbench_api.py code to modify and set the model you are using.

```
:~/YOUR_FOLDER/MAgentBench/MAgentBench$ ls
# here you will found mlagentbench_api.py
# Also in VS Code it would be easy to find, in the same folder
# with the folder of agents, benchmarks, LLM.py and code plots
```

Here the only change to made is the name of the model as you can see below

```
llm_name: str = Field("llama-3.1-8B-Instruct", description="--llm-name")
# Change models here
fast_llm_name: str = Field("llama-3.1-8B-Instruct", description="--fast-llm-name")
edit_script_llm_name: str = Field("llama-3.1-8B-Instruct", description="--edit-script-llm-name")
#"qwen2.5-7B-Instruct"
```

- **FastAPI -uvicorn:** Thanks to the implementation of FastAPI and uvicorn, we can establish communication with our container so that it can receive requests in the command format we saw earlier to run each task. It will send this format to the vllm, but first we need to establish communication.

```
uvicorn MAgentBench.mlagentbench_api:app --host 0.0.0.0 --port 8000
```

Launch the API server in a new bash within the container again.

- **JMeter:** Here, JMeter acts as a load generator and traffic simulator, designed to send multiple requests to our API server and simulate multiple users.

Here, Jmeter is designed to simulate three users (you can have as many as you want, but initially three) to use the two models and two different types of response format that the model has to perform.

```
"task": "cifar10",&#xd;
"log_dir": "user1_cifar10_llama/user_${__threadNum}",&#xd;
"work_dir": "workspace/cifar10_llama/user_${__threadNum}",&#xd;
"llm_name": "llama-3.1-8B-Instruct",&#xd;
"device": 0,&#xd;
"agent_type": "ResearchAgent",&#xd;
"max_steps": 50,&#xd;
"valid_format_entires": [&#xd;
  "Thought",&#xd;
  "Action",&#xd;
  "Action Input",&#xd;
],&#xd;
"max_steps_in_context": 1,&#xd;
"max_observation_steps_in_context": 1,&#xd;
"max_retries": 8,&#xd;
"use_codecarbon": ${__P(use_codecarbon,false)}&#xd;
}&#xd;
```

Inside the JMETER folder, you will see that there is a .jmx file for each task, distinguished by the name **"task"_users.jmx** (rigid response format) and others **"task"N_users.jmx** (original response format), which is provided so that comparisons can be made.

In this small fragment of one of these tasks, more specifically one that is designed for the rigid response format.

You can see how the working folders and the folders where the files will be saved are defined. It is also configured so that if more than one user is used, they will be saved within (for example, user1_cifar10_llama/user_1 or user_2 or user_3).

The only recommendation here is, depending on the model being used, to change the name of **llm_name** and the **folders**. For example, in the case of using **Qwen 2.5**, it would be 'user1_cifar10_qwen/'. This is for the purposes of order and reproducibility later on.

Command to launch Jmeter and number of users: In a new bash, not inside the container, where the host and your cloned folders from the repository are located, apply the following:

```
jmeter -n -t JMETER/cifar10_users.jmx \ # File used
-l JMETER/results_.jtl \ #save the results as the name you prefer
-Jusers=1 \ # Define the number of users
-Juse_codecarbon=true # The same flag to enable the use of Codecarbon
```

This will run and you will see it executing in the bash. You will have to wait until the option to execute commands becomes active there.

6. Evaluation and Visualization

Finally, we move on to the last stage, which is the evaluation and plotting of the graphs. With regard to each of the runs made by each user and each model, for simplicity's purposes, only two will be shown.

- **Evaluation:**

As seen earlier in section 4, we run the command to evaluate the task.

```
python -m MLAGentBench.eval \
--log-folder user1_cifar10_llama/user_1 \
--task cifar10 \
--output-file user1_cifar10_llama/user_1/resultados_llm.json
```

Specify the exact path where resultados_llm.json is located.

- **Plots:** Below are the commands for plotting each of the analysis graphs.

This command will generate bar graphs showing the energy consumption of each step of the model, as well as a graph showing the total energy consumption of the tools used in the process.

```
python MLAGentBench/plot_codecarbon.py --exp-dir user1_cifar10_llama/user_1
# This will save your codecarbon plots in the same folder you use
# With the name "codecarbon_plots"
```

If you want to compare energy consumption between tasks or between models, you are free to choose what to compare using this command.

```
python MLAGentBench/plot_codecarbon.py --compare \
./user1_babyllm_llama/user_1 ./user1_babyllm_qwen/user_1 \
./user1_cifar10_llama/user_1 ./user1_cifar10_qwen/user_1 \
```



```
./user1_clrs_llama/user_1 ./user1_clrs_qwen/user_1 \  
--out-dir ./mis_multi_plots # it will save here the plots
```

This is an example of a comparison of three tasks using the two models.

This command is used to view the accuracy against time of several runs. It can be applied to any task, but in this case, the average of the same model with the same task is being applied to view the variability of success.

```
python -m MLAGentBench.plot_accuracy_time \  
--runs \  
user1_cifar10_llama/user_1 \  
user1_cifar10_llama/user_2 \  
user1_cifar10_llama/user_3 \  
--metric-name "CIFAR10 test accuracy" \  
--avg-lines \  
--out mis_multi_plots/cifar10/cifar10_time_vs_score_avg.png  
# Recommended to create the folder where you want to save the file "--out"  
# If you do not want to see the average, simply remove the '--avg-lines' flag.  
# Also you can change the name of the metric as your desire.
```

- **Export data to Excel:** Finally, when the necessary or desired runs have been completed, this command will search for all results_llm.json files to organise the information, generally three users per task. If not all are the same, it will do so, but will only display a warning message. It will export an Excel file with the following relevant information for comparison and analysis of the tests: Score_per_task, Completion_rate, Energy_consumption, and a global summary.

```
python -m MLAGentBench.results_llm
```

It does this by distinguishing between models and response format types (rigid or normal), which is why JMETER leaves those folder names so that the code can find them.

7 Configuration agent_research.py

It's simple here, just uncomment a few lines of code that should go inside

INVALID_RESPONSE_INSTRUCTIONS =

Message found in the first lines below **format_prompt_dict =**

```
INVALID_RESPONSE_INSTRUCTIONS = ""  
Response is invalid and discarded.
```

You MUST answer in EXACTLY this format:

Thought: <your reasoning in one or two sentences>

Action: <ONE tool name from the list below>

Action Input: <a VALID JSON object with the arguments for that tool>

Do NOT invent new action names like "Research Plan and Status" or
"the action to take, should be one of the names of the tools".
Those strings are instructions, NOT tools.

The Action line MUST be exactly one of the tool names above.

The Action Input MUST be a valid JSON object.

If we want to use the response format more strictly,

#we can add these lines to the instructions:

#Do NOT add extra sections or headings. Only:

Thought:

Action:

Action Input:

"".strip()

If we want to use the original response format with more sections,

we can add these lines to the instructions:

Do NOT add extra sections or headings beyond these six required sections.

The idea is to remove those **thought:**, **action:**, and **action input:** from those invalid response instructions, and insert the commented lines below, which will enable the model to respond with the original format.